

Raj Kumar Nelluri

(201) 554-8009 | rajkumarn2002@gmail.com | [LinkedIn](#) | [GitHub](#) | [Portfolio](#)

PROFESSIONAL SUMMARY

AI/ML Engineer specializing in production-grade LLM systems, RAG pipelines, and computer vision — deployed live on AWS. Experienced building end-to-end AI applications using OpenAI APIs, LangChain, LangGraph, FastAPI, and Docker. Two systems currently live on custom subdomains on AWS EC2. Strong focus on deterministic decision systems, hallucination mitigation, scalable MLOps, and cloud-native deployment.

TECHNICAL SKILLS

Programming: Python, SQL

AI / LLM Systems: Retrieval-Augmented Generation (RAG), LangChain, LangGraph, OpenAI API, Prompt Engineering, Embeddings, Vector Databases (ChromaDB), Hugging Face, Deterministic Decision Systems

Machine Learning: Scikit-learn, XGBoost, TensorFlow, PyTorch, YOLOv8, ResNet-50, LSTM, Prophet, SHAP

Backend & APIs: FastAPI, REST APIs, Redis, Streamlit

Cloud & Deployment: AWS (EC2, SageMaker, Lambda, S3, Kinesis, RDS, CloudWatch, EventBridge), Docker, Nginx, GitHub Actions

Data & Analytics: Pandas, NumPy, Matplotlib, MLflow, Tableau, Power BI

AI ENGINEERING PROJECTS

AI Financial Research Agent | [Live Demo](#) | [GitHub](#)

2026

FastAPI | LangGraph | GPT-4o | Alpha Vantage | Redis | Docker | AWS EC2 | Nginx

- Built a production-grade investment analysis system generating deterministic BUY/HOLD/SELL recommendations in under 30 seconds by integrating live market data, company fundamentals, and news sentiment
- Engineered normalized multi-factor scoring engine (Technical 0–25, Fundamental 0–40, Sentiment 0–15) with dynamic max adjustment — missing data excluded from denominator rather than penalised, eliminating false SELL signals
- Developed conflict-aware decision logic detecting signal disagreement (variance > 0.15) and overriding to HOLD in 35–55% ambiguity band, reducing false conviction on mixed signals
- Constrained GPT-4o to explanation-only role with no influence over BUY/HOLD/SELL decision, ensuring fully reproducible and auditable outputs — zero LLM hallucination risk on recommendations
- Implemented 5-factor confidence model (data completeness, signal agreement, volatility, news uncertainty) preventing recommendations below 20% confidence threshold
- Architected 7-step async LangGraph pipeline on AWS EC2 with Redis caching (TTL 900s), reducing repeated query latency from ~15s to <100ms — 10× API cost reduction
- Deployed multi-service architecture with Nginx domain-based routing, Docker Compose, and portfolio ranking mode supporting 2–8 tickers with proportional capital allocation

Enterprise RAG Chatbot — AI Document Intelligence | [Live Demo](#) | [GitHub](#)

2026

Python | LangChain | OpenAI API | ChromaDB | Streamlit | Docker | AWS EC2 | Nginx

- Built a full-stack RAG chatbot from scratch — PDF ingestion, vector search, LLM response, and deployed it on AWS EC2 behind nginx with HTTPS using Let's Encrypt
- Shipped two separate interfaces: a Streamlit UI for end users to chat with their documents, and a FastAPI Swagger UI at /api/docs for developers to explore and test all endpoints directly
- Wrote a PDF ingestion pipeline that loads documents, splits them into 1000-character chunks with 200-character overlap using RecursiveCharacterTextSplitter, embeds them with OpenAI text-embedding-3-small, and stores them in a persistent ChromaDB collection
- Used MMR (Maximal Marginal Relevance) retrieval instead of plain cosine similarity to fetch a larger candidate pool and re-rank for both relevance and diversity, preventing the LLM from seeing near-duplicate chunks
- Built a rule-based query classifier that detects query intent — factual, complex, ambiguous, or keyword — and adjusts retrieval parameters like `top_k` and `fetch_k` accordingly without any extra LLM call

- Added hallucination guardrails that check retrieval confidence before calling the LLM — if no chunks are returned, scores are too low, or context is too short, it returns a fallback message instead of guessing
- Implemented Redis caching with query normalization (strips punctuation, lowercases, removes filler phrases like "can you") so semantically identical queries hit the cache and skip the OpenAI API entirely
- Added three cost control guards on the API endpoint — rate limiting via slowapi (20 req/min per IP), input length cap (500 chars), and token budget enforcement using tiktoken — all firing before any LLM call
- Structured prompts with explicit [SOURCE N: filename — Page: X] citation format so the LLM is forced to cite every fact from the retrieved context, not its training data
- Tracked p95 response times, cache hit rates, retrieval times, and document access patterns with a custom MetricsTracker, and logged every query as structured JSON for easy analysis
- Containerized the full stack (Streamlit, FastAPI, Redis) with Docker Compose and built a safe deploy workflow — build locally on Mac, push to Docker Hub, EC2 pulls — since the t3.micro server can't handle image builds

Customer Churn Prediction — MLOps Pipeline | [GitHub](#)2025

Python | XGBoost | AWS SageMaker | Lambda | CloudWatch | EventBridge | MLflow | Docker

- Built end-to-end MLOps pipeline achieving 91% accuracy and 89% F1 score: S3 ingestion → Lambda ETL → SageMaker training → real-time inference endpoint with 21 engineered features
- Integrated AWS Comprehend NLP to extract sentiment signals from 7k+ customer support tickets as additional churn predictors
- Implemented automated model retraining via CloudWatch + EventBridge triggers on data drift detection; MLflow tracks all experiments with full reproducibility

ADDITIONAL PROJECTS

Retail Sales Forecasting System | [GitHub](#) | *LSTM, Prophet, PyTorch, AWS SageMaker, S3*2025

- LSTM + Prophet ensemble achieving 4.2% RMSE across 45 product categories — 18% improvement over single-model baselines on 500,000+ retail transactions with automated weekly SageMaker retraining

Insurance Fraud Detection System | *YOLOv8, ResNet-50, PyTorch, FastAPI, AWS Lambda, Kinesis, RDS*2024

- Dual-model CV pipeline: YOLOv8 for vehicle damage localisation + ResNet-50 for severity classification achieving 96.7% accuracy on 50,000+ claims at 50ms inference latency via FastAPI
- Implemented real-time data ingestion using AWS Kinesis, serverless processing with Lambda, and fraud score storage in RDS enabling SQL-based analyst queries

EDUCATION

Pace University	New York City, NY
<i>Master of Science in Computer Science</i>	<i>Sep 2023 – Apr 2025</i>
Amrita Vishwa Vidyapeetham	Bengaluru, India
<i>Bachelor of Technology in Artificial Intelligence</i>	<i>Jul 2019 – Apr 2023</i>

CERTIFICATIONS

AWS Certified Cloud Practitioner – Amazon Web Services	2025
Adobe Journey Optimizer Foundation – Adobe	2025
Complete Python Developer – Udemy	2023